

FFNN-SGD

etawn

1 Introduction

This is a somewhat garbage implementation of a Feedforward Neural Network trained via Stochastic Gradient Descent. This write-up just describes the inner workings of the program and can serve as a reference, although using this as a reference would be a bad idea. Gives me more reason to larp as well.

2 Definition of Terms

Epoch: One cycle of training the neural network with the entire training dataset.

Gradient: The direction of the steepest increase or decrease.

Loss Function: A function that tells the neural network how **wrong** its predictions are.

Cost: The average or sum of the loss across many samples.

Neuron: An element that holds data (typically a single scalar value) in a network.

Layer: A group of neurons that sit at the same depth of a network.

Activations: The actual values of the neurons after processing their inputs.

3 MNIST Dataset

This implementation uses the MNIST dataset which contains several 28×28 images used as training and test data.

More information about the dataset can be found on Kaggle:
<https://www.kaggle.com/datasets/hojjatk/mnist-dataset>

4 Feedforward Neural Network

The simplest Neural Network architecture consisting of 3 layers and information flowing in one direction.

4.1 Layers

Input Layer: Where raw data from the MNIST dataset is passed into.

Hidden Layer: Each neuron in this layer takes the previous layer's output as input, computes its own weighted sum, adds a bias, applies an activation function, and passes its result forward to future layers.

Output Layer: Produces the final one-hot encoded column-vector containing the predictions of the neural network.

When data passes through a neuron, it calculates the weighted sum and the activation for that neuron.

The weighted sum z is defined as:

$$z = Wx + b \quad (1)$$

where W represents the weights, x is the input vector, and b is the bias value.

We take that weighted sum and pass it through an activation function depending on the layer we are currently at. The last layer uses a Softmax function while the others use a ReLU (Rectified Linear Unit).

The final activation a is calculated by passing the weighted sum z through an activation function f :

$$a = f(z) \quad (2)$$

where:

- a is the output of the neuron (activation).
- f represents the activation function (such as ReLU, Sigmoid, or Tanh).
- z is the weighted sum ($Wx + b$).

There are several activation functions; however, in this implementation we only use **ReLU** and **Softmax**.

5 Activation Functions

5.1 ReLU

Rectified Linear Unit or ReLU is an activation function which is defined by the following equation:

$$a = \max(0, z) \quad (3)$$

Basically, we take a column-vector and convert all its negative values to 0.

5.2 Softmax

The Softmax function takes a column-vector and squishes its values so that they all lie between 0 and 1 and sum up to 1.0. This is used to output a clean probability distribution across all labels.

6 Stochastic Gradient Descent

Imagine yourself as a blindfolded ball on a somewhat scuffed mountain containing multiple peaks and valleys. Your goal is to reach the absolute lowest point of this mountain (global minimum) which gives the lowest cost of the loss function. Considering that we cannot see, it is pretty hard to reach the absolute lowest point, which means we have to settle with what we can reach for now, which is the local minimum.

Regular Gradient Descent does this by computing the gradient of the entire dataset and taking a step in that direction each time. This is horrible computation-wise because datasets can be huge.

Stochastic Gradient Descent uses the idea of computing the gradient of a small batch of the dataset, and then stepping in that direction until the entire dataset is exhausted.

Since both of them exhaust the entire dataset in one epoch anyway, why not just use Regular Gradient Descent? Well, Regular Gradient Descent steps once per epoch, which means it takes a long time for the neural network to actually make meaningful predictions, while Stochastic Gradient Descent can step multiple times in a single epoch.

To minimize the cost of the loss function, we basically want to know how and where to nudge the weights and biases of our network. This is done through Backpropagation which will be discussed further down the write-up.

7 Implementation

7.1 `matrix.c`

Contains matrix functions. Not much to say here.

7.2 `mnist_loader.c`

Parses the MNIST dataset and loads them into a `MNIST_DS` struct containing pixel information and dimensions.

The MNIST dataset uses a `.idx` file format. More information about it can be found here:

https://www.fon.hum.uva.nl/praat/manual/IDX_file_format.html

This file also contains an implementation of the **Fisher-Yates Shuffle Function** to shuffle the dataset in preparation for SGD.

7.3 `activations.c`

Contains the different activation functions.

7.4 network.c

This is the actual neural network implementation. It contains initialization functions, the Forward Pass and Backpropagation.

It also contains an implementation of Xavier Glorot’s Weight Initialization function to initialize the weights of each neuron for the current layer’s weight matrix.

7.4.1 Uniform Xavier Glorot Initialization

This function initializes a weight matrix with arbitrary float values within the range $[-x, x]$ where x is defined by the equation:

$$x = \sqrt{\frac{6.0}{N_{\text{in}} + N_{\text{out}}}} \quad (4)$$

where N_{in} is defined as the number of input neurons and N_{out} is the number of output neurons.

7.4.2 Forward Pass

This function performs the neural network’s process of computing weighted sums and biases and applying activation functions to pass onto the next layer until the output layer is reached.

The output layer’s post-activation column vector contains the neural network’s predictions.

7.4.3 Backpropagation

Backpropagation is the algorithm that helps compute the gradients that make the cost of the loss function smaller. As mentioned in the SGD section, we need these gradients for the SGD to actually apply these changes during training. We basically compute how wrong our network’s prediction was and propagate that to the previous layers.

To know the correct bias values, we need to compute the bias gradient. Fortunately, the bias gradient is directly equal to the current error term (delta value).

For the weight gradient, we need to look at the inputs they were given during the forward pass. The weight gradient is defined as:

$$\frac{\partial L}{\partial W_{ij}} = \delta_i \cdot a_j \quad (5)$$

where:

- δ_i is the error term of the current neuron (i).
- a_j is the activation value sent by the previous neuron (j).

Basically, the delta of the current neuron is multiplied by the previous activation values.

We require the previous activation values matrix to be transposed for proper matrix multiplication.

If we have not reached the input layer yet, we calculate a new delta for the previous layer to scale the error according to the weight values that caused them.

To understand this a bit better, if we multiplied weights to send a signal forward, we must multiply by the transposed weights (W^T) to trace the error backward.

Since we are tracing errors backward, we obviously do not want to pass errors to neurons that are off. We can identify these neurons to be the ones that have a value of 0 during the forward pass, due to ReLU converting them to 0. Due to them being 0, we can infer that they have no effect on the calculations of the later layers. This would mean this neuron has no responsibility for the final error. Hence, we perform a ReLU Derivative function to protect this neuron from being blamed for the error.

7.5 train.c

This handles the neural network's training process. It includes one-hot encoding, cross entropy, performing the SGD and an evaluation process per epoch.

7.5.1 One-Hot Encoding

This function takes a column vector and a label. We basically want to set all the values of the column vector to be 0 and then set whatever index of label is in that column vector to be 1. This is basically the desired output vector.

7.5.2 Cross Entropy

Cross Entropy basically takes two probability distributions and returns a value that shows how far apart they are.

In the implementation we just take the negative logarithm of the network's predicted value for a given label. A small cross entropy value would mean our network is highly confident on the right answer, while a high cross entropy is confident on the wrong answer, or is quite unsure about the right answer.

8 Conclusion

And that concludes all the larping. STM tuition did not go to waste :)